

Image Processing Using C/C++

1 - Introduction

In this project, you will be doing some image processing using C/C++. You will be required to implement the following image processing operations.

- *Histogram Stretching*
- *Block Truncation Coding*
- *Edge Detection*

2 - Background

2.1 - PGM Files

Images used in this project will be provided in PGM (Portable Gray Map) because of their ease of reading and writing. Code is provided to handle reading and writing of the PGM files where all image processing procedures will be done on images that have been loaded in memory of your program. The provided functions for reading/writing are written in a C-style format since more complex C++ methods have yet to be presented in the course.

A PGM file is in grayscale image format. Each file contains a header specifying information about the file followed by the image data itself. Below is a PGM file example (reference: <http://netpbm.sourceforge.net/doc/pgm.html>).

```
P2
# feep.pgm
24 3
15
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 3 3 3 3 0 0 7 7 7 7 0 0 11 11 11 11 0 0 15 15 15 15 0
0 3 0 0 0 0 0 7 0 0 0 0 0 11 0 0 0 0 0 15 0 0 15 0
```

One of the easiest ways to open and view a PGM file is to use an online PGM viewer, such as <https://bytes.usc.edu/~saty/tools/PGMViewer/viewer.html>.

2.2 - Histogram Stretching

Histogram stretching improves the contrast of an image by expanding the range of pixel values. It detects the minimum pixel value (min) and the maximum pixel value (max) of an original image and converts the pixel range from [min, max] to [0, 255]. The formula for histogram stretching is:

$$\text{output_pixel_value} = 255 \times (\text{original_pixel_value} - \text{min}) / (\text{max} - \text{min}).$$

2.3 – Block Truncation Coding (BTC)

BTC is a lossy image compression algorithm. The basic idea is to divide an image into non-overlapping 4×4 blocks and represent each block using three pieces of information: 1) a 1-bit bitmap indicating whether each pixel is below or above the average pixel value of the block; 2) a low reconstruction level, representing the average value of the pixels classified as 0 in the block; and 3) a high reconstruction level, representing the average value of the pixels classified as 1 in the block. For example, suppose that we have the following 4×4 block.

100	102	105	110
98	101	107	112
95	100	108	115
90	97	110	120

The average of the 16 pixel values is 104 (**rounded down**). Next, every pixel is compared with the average value. A pixel is represented by 1 if its value is **greater than or equal to** 104, and by 0 otherwise.

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

Reading the bitmap from left to right and top to bottom, the block is therefore represented by the 16-bit bitmap: 0011001100110011.

In addition to the bitmap, two reconstruction levels are calculated: the low reconstruction level (the average of the pixels represented by 0) is calculated as 97 (**rounded down**); and the high reconstruction level (the average of the pixels represented by 1) is calculated as 110 (**rounded**

down). Thus, the compressed representation of this 4×4 block consists of 0011001100110011 + 01100001 (=97₁₀) + 01101110 (=110₁₀), and the data size is reduced from 4×4×8 bits to 16+8+8 bits, which means that the compression ratio is 4 (=128/32).

To reconstruct (decompress) the image, each 0 in the bitmap is replaced with the low reconstruction level (97), and each 1 is replaced with the high reconstruction level (110). Therefore, the reconstructed block is as follows.

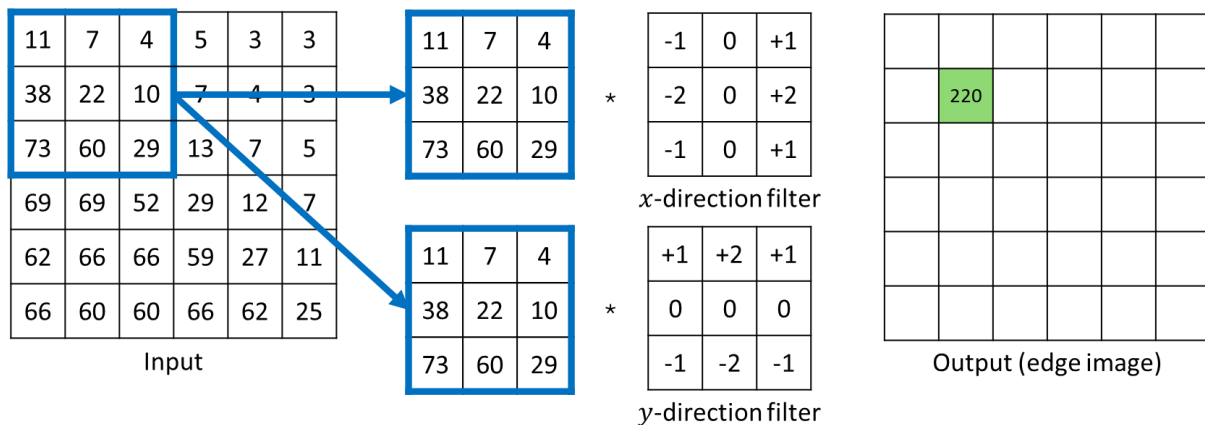
97	97	110	110
97	97	110	110
97	97	110	110
97	97	110	110

For this project, you will use BTC to generate a reconstructed image from a given input image. Note that you are not required to output the compressed bitstream. The input image should be divided into non-overlapping 4×4 blocks, with each block reconstructed independently.

2.4 - Edge Detection

Edge detection is used to identify edges within an image, which represent areas where there is a notable change in pixel intensity. This process is fundamental for various image processing tasks, such as object detection, image segmentation, and feature extraction.

In this project, the Sobel edge detection algorithm is used. It utilizes two 3×3 constant filters (also known as kernels) to detect edges: one for horizontal edges and the other for vertical edges. The algorithm works by convolving each filter with the image. As each filter slides over the image, it computes a weighted sum of pixel values, G_x and G_y . To determine the overall edge strength, G_x and G_y are combined as $G = \sqrt{(G_x)^2 + (G_y)^2}$, which represents an edge intensity of a pixel. **To avoid overflow, if the G value exceeds 255, it is capped at 255.** Note that applying thresholding improves the quality of the resulting edge image, but it is not used in this project for the sake of simplicity. An example of this process is shown in Figure 1.



$$G_x = 11*(-1) + 7*0 + 4*1 + 38*(-2) + 22*0 + 10*2 + 73*(-1) + 60*0 + 29*1 = -107$$

$$G_y = 11*1 + 7*2 + 4*1 + 38*0 + 22*0 + 10*0 + 73*(-1) + 60*(-2) + 29*(-1) = -193$$

$$\text{Min}(255, \sqrt{(G_x)^2 + (G_y)^2}) = 220$$

[Figure 1. Edge detection example. The 3x3 windows slide over the original image by one pixel in a Z-order (from left to right; from top to bottom).]

For this project, the boundaries of the resulting image (i.e., the first row, the last row, the first column, and the last column) are to be filled with the original values for simplicity. The 3x3 filter windows slide not only from left to right but also from top to bottom. The filters you will be using for this project are the same as those shown in Figure 1. These filters are already implemented in the edgeDetect() function.

You can use the sqrt() function from the cmath standard library, which returns a double-typed value. To convert the result into an int-typed value, you can use the following syntax: "int-typed variable = static_cast<int>(sqrt() result);".

3 - Tasks

Implement the **histogramStretching()**, **btc()**, and **edgeDetect()** functions in the ImageProcessing.cpp file.

- You can find information about the parameters and return data of these functions in the ImageProcessing.h file.
- **To match the precision of golden images with your resulting images, do not use the *double* and *float* data types (use the *int* data type).**
- PGM.cpp and PGM.h are used to read and write PGM files. There is nothing to do with these files.
- main.cpp receives the name of an input image, the type of operation (histogram stretching, btc, or edge detection), and the name of an output image. There is nothing to do with this file.
- You can define additional functions in ImageProcessing.cpp if you want.
- Do not use any built-in functions except for sqrt()

4 - Compile and Test

```
g++ -o main *.cpp  
./main
```

- ***Histogram Stretching***

- Inputs to the program

Enter Original File Name: **input_images/balloons.pgm**

...

Enter Selection: **0**

...

Enter Save File Name: **output_images/stretching.pgm**

- Original and resulting images



- **Block Truncation Coding**

- Inputs to the program

Enter Original File Name: **input_images/cameraman.pgm**

...

Enter Selection: **1**

...

Enter Save File Name: **output_images/btc.pgm**

- Original and resulting images



- **Edge Detection**

- Inputs to the program

Enter Original File Name: **input_images/cameraman.pgm**

...

Enter Selection: **2**

...

Enter Save File Name: **output_images/edgedetect.pgm**

- Original and resulting images



- ***Check all together***

```
chmod 777 run.sh  
./run.sh
```

You must see the following messages:

```
HISTOGRAM STRETCHING - PASS  
BLOCK TRUNCATION CODING - PASS  
EDGE DETECTION - PASS
```

5 - Submit

- Please upload your **ImageProcessing.cpp** file to **myCourses > Assignments > Project1**.
- Do not modify other files or create new files so that the grader can easily grade your submissions.
- Do not submit *.docx, *.pdf, *.txt, or *.zip file.

6 - Grading

- [90 pts] - Functions to be implemented
 - [20 pts] histogramStretching()
 - [35 pts] btc()
 - [35 pts] edgeDetect()
- [10 pts] - Program execution
 - **The program execution portion of the grade is ALL or nothing.**